Compiled spatial optimisation: exact resident state and GPU population search for landscape harvest scheduling

UBC FRESH Lab

November 3, 2026

Abstract

Spatially explicit forest planning models are difficult not because the objective is exotic, but because the model, the exact accounting state, and the search move are usually rebuilt together. This paper describes a compiled-state architecture that separates four concerns: a stable compiled model, compact candidate and sparse effect indexes, exact resident accounting, and batched GPU population search with exact CPU replay. A CPU-refinement handoff is designed but is not part of the measured campaign. We evaluate the architecture on a public frozen TSA24 landscape with 208,255 operable blocks. A 1,000-generation GPU search reduces the weighted penalty from 1,722,144,291.336651 to 8,588,725.370498 in 9.381 s of search time; a 100,000-generation, five-seed campaign reaches 512.574–518.621 (mean 515.682) with exact CPU replay. The target uses deliberately unattainable harvest-level aspirations to pull the solution toward the maximum even-flow level; the final level is approximately 51.21–52.49M m^3 per 10-year period against a 70M aspiration. The frozen dataset, target, and seed are public; the paper-specific schedules and replay scripts are retained in the project evidence repository and are not yet publicly released. We do not claim an independent model comparison or a state-of-the-art landscape result; the contribution is the architecture and its reproducibility contract.

1 Introduction

Forest planning models combine long planning horizons, spatially explicit treatment choices, and objectives that mix timber production with ecological and operational constraints. At the scale of a timber supply area, the decision is not a single harvest level but a schedule: which blocks receive which treatments, in which periods, under which spatial and temporal rules. The combinatorial state is large, the feasible set is structured, and the objective is usually a weighted compromise among targets. The resulting models are hard for exact methods and expensive for generic heuristics.

A recurring engineering problem sits underneath the optimisation problem. A compiled forest model contains blocks, fragments, strata, tracks, treatments, transitions, growth curves, products, and accounting rules. A search move changes one small part of that model, but the implementation often recomputes or reconstructs a much larger state to evaluate it. The model, the accounting state, and the move are therefore rebuilt together. This coupling makes experiments slow, makes replay fragile, and makes it difficult to separate the effect of a search method from the effect of an implementation detail.

This paper asks a narrower question than "which metaheuristic is best for forest planning?" It asks whether a compiled-state architecture can make a large landscape search exact, replayable, and fast enough to support controlled experiments. The architecture has five layers:

1. a compiled model;

2. a compact candidate index;
3. a sparse effect table over target cells;
4. exact resident accounting state;
5. a GPU population search with exact CPU replay; a CPU-refinement handoff is designed but not measured here.

The first two layers make the model cheap to query. The sparse table makes an evaluation proportional to the cells a move touches rather than to the whole schedule. The resident state makes the objective authoritative and reversible. The design loop uses the GPU for breadth and the CPU for exact accounting and local refinement; the measured campaign uses GPU search with exact replay and does not include the refinement arm.

We evaluate the architecture on a frozen TSA24/Prince George even-flow instance. The instance has 208,255 operable blocks out of 458,708 compiled blocks and a 20-period horizon. The target is a weighted goal-programming construction: a dominant even-flow target defines the shape, while deliberately unattainable harvest-level aspirations provide a persistent pull toward more volume. The realized level is an outcome, not a declared feasible floor. This construction is intentional; the 70M m^3 per-period aspiration is not a hard constraint and the reported residual is not evidence that the model is infeasible.

The paper makes three contributions:

- a compiled-state architecture that separates model, accounting, and search;
- an exact replay contract for a large landscape instance, including a public frozen artifact;
- a measured scaling record from 1,000 to 100,000 GPU generations, with five-seed evidence at the largest budget.

It does not make a competitive landscape claim. The CPU simulated-annealing comparison reported here is a weak, untuned secondary comparator, and no independent harvest-scheduling model is available on the same instance. Those limits are part of the result, not a footnote.

2 Related work

2.1 Spatial harvest scheduling

Spatial harvest scheduling has a long history in forest operations research. Lockwood and Moore [1993] formulated harvest scheduling with spatial constraints and applied simulated annealing to a problem in which adjacency and opening-size restrictions make the feasible set non-trivial. Their work established a template that remains recognizable: represent a schedule as a set of treatment decisions, define a penalty or objective over the resulting landscape, and use a stochastic search to move between feasible or penalized states.

Boston and Bettinger [2002] compared tabu search and a genetic algorithm on spatial harvest scheduling problems. Their comparison is important here because it treats the search method as an experimental factor rather than as an implementation detail. Bettinger et al. [2002] extended the comparison to eight heuristic planning techniques on three wildlife planning problems of increasing difficulty. Together, these studies show that the choice of neighbourhood, the representation of the schedule, and the handling of spatial constraints can matter as much as the name of the metaheuristic.

The present paper is not a replacement for those comparisons. It addresses the layer beneath them: how the model state and objective are maintained while a search runs. The architecture is

designed to make that layer explicit, exact, and replayable, so that future comparisons can vary the search method without rebuilding the model and accounting machinery.

2.2 Exact state and neighbourhood search

Metaheuristic implementations often trade exactness for speed. A move is evaluated by an approximate delta, a cached value, or a full re-simulation; each choice changes what the search is actually optimising. For forest planning, this is a material issue because the objective is a weighted goal-programming penalty over multiple accounts and periods. An approximate evaluator can make a schedule look better than it is, and a full re-simulation can make a controlled experiment impractically slow.

The architecture described here keeps the exact objective as the authority. A move is priced by the cells it changes, but the resident state is updated with the same accounting rules used by the full evaluator. The search can therefore use a fast delta without giving up the exactness of the reported incumbent. This is closest in spirit to the exact incremental evaluation used in move-based local search, but it is applied to a compiled landscape model rather than to a generic permutation problem.

2.3 Reproducibility and software infrastructure

Reproducibility in applied optimisation is usually discussed at the level of random seeds and reported objective values. That is necessary but not sufficient. A result also depends on the compiled model, the candidate order, the effect table, the treatment definitions, the objective weights, the hardware, and the replay procedure. If those are not part of the artifact, a reader cannot tell whether a difference comes from the algorithm or from the data pipeline.

This paper treats the evidence record as part of the architecture. Each reported schedule is stored with its objective, seed, configuration, hardware, and a hash. An exact replay loads the schedule through the same model and evaluator, rather than trusting the search process that produced it. The frozen TSA24 artifact and its rebuild recipe are published so that the GPU result is not a local demonstration that cannot be inspected.

The related work on reproducible optimisation software is broad, and this paper does not attempt a general survey. Its narrower claim is that an exact resident state plus an explicit artifact can make a large landscape experiment auditable even when the search itself is stochastic.

3 Problem and model

3.1 Instance

The case study uses the TSA24/Prince George landscape compiled for WS4. The compiled dataset contains 458,708 blocks; 208,255 of them have at least one operable treatment chain and therefore form the search universe. The horizon is 200 years, divided into twenty 10-year accounting periods. The model includes blocks, fragments, tracks, strata, treatments, growth and yield curves, features, products, and account definitions. The schedule assigns a treatment chain to each operable block. An empty chain is a valid decision: a block may be left untreated.

The instance is frozen and published as `ws4-tsa24-evenflow-v1` with the compiled dataset, target file, initial schedule, license terms, and a checksum manifest. The artifact is a derived benchmark bundle; the raw BC source layers retain their own open-government terms.

3.2 Even-flow target and level aspirations

The objective is weighted goal programming. Each target contributes a one-sided penalty for falling below a minimum or exceeding a maximum:

$$s_{t,p} = \max(m_t - v_{t,p}, 0), \tag{1}$$
$$e_{t,p} = \max(v_{t,p} - M_t, 0), \tag{2}$$
$$\tau_{t,p} = w_t^- \, \phi(s_{t,p}) + w_t^+ \, \phi(e_{t,p}). \tag{3}$$

The shape is linear or quadratic. The target set is not a set of hard constraints. The names `minimum` and `maximum` name the hinge of the penalty, not a feasible interval.

The TSA24 target has two roles. First, one even-flow target spans periods 1–20, compares each period with the reference period, and charges a squared deviation outside a $\pm 1\%$ band with weight 10,000. Second, every period carries a linear harvest-level target with aspiration 70,000,000 m^3 per 10-year period and weight 100. The level aspiration is deliberately above the level the estate is expected to achieve. It is a pull-up device: every period has a downward gradient toward more harvest, including periods that would otherwise be left underused. The even-flow target selects the flattest profile at the level where the two roles reach their best weighted compromise. The realized level is therefore an outcome, not a declared floor.

This construction is the maximum-even-flow pattern documented for WS4. It is not an infeasibility error, and a large residual on the level term is not evidence that the target should be removed. It is the expected consequence of an aspirational target that creates a search gradient. The paper reports the split between the flow and level terms, the realized harvest level, and the target values so that a reader can inspect the compromise rather than infer it from a single total.

3.3 Initial schedule and feasibility

The initial schedule is the oldest-first seed from the public artifact. It is a valid block/treatment schedule and therefore a feasible starting point for the search, but it is not an optimized control. The initial objective is 1,722,144,291.336651. The very large initial penalty is dominated by even-flow deviations; the seed is a starting point chosen for reproducibility, not a baseline chosen to make the search look effective.

The model has no hard spatial constraints in this run. The target residuals are soft, and the reported schedule is a compromise. The paper therefore distinguishes schedule validity from target satisfaction: the schedule is feasible, while the targets are aspirations.

3.4 Objective and evaluation

For a schedule `S`, let `v(S, p, a)` be the realized value of account `a` in period `p`. The objective is

$$Z(S) = \sum_t \sum_{p \in P_t} \tau_{t,p}\big(v_{t,p}(S)\big). \tag{4}$$

The implementation evaluates this objective exactly. A search move may use a cached per-block contribution to price its effect, but the resident state is updated with the same accounting rules as the full evaluator. The reported objective is never an approximation produced by the search loop. This distinction is central to the evidence contract: a fast delta is allowed, but the incumbent must replay through the exact evaluator.

4 Architecture

4.1 Compiled model

The compiled dataset is the boundary between ingestion and execution. It is a versioned directory of columnar tables plus a manifest. The tables describe fragments, blocks, tracks, strata, treatments, curves, features, products, accounts, groups, and compile diagnostics. The manifest records the schema version, source identities and hashes, compile options, and table list.

Compilation happens at the fragment level; a block is a runtime aggregate of its fragments. The separation matters because fragments carry the inventory attributes, while the search and reporting interfaces operate on blocks. The search never reads the raw inventory or recompiles the recipe.

4.2 Candidate index

For each block, the candidate index stores the feasible treatment chains in a stable order. A chain is an ordered sequence of treatment events `(treatment, year)`. The order is part of the contract: candidate indices are referenced by the effect table and by the search, so changing the order changes the meaning of a cached effect.

The index is built by walking track transitions from the block's initial state and applying the treatment eligibility and operability windows. It includes the empty chain. The index is deliberately smaller than the full cross-product of treatments and years, but it remains richer than a single "harvest/no harvest" decision. The search can toggle a chain, shift an event, swap chains between blocks, or apply a larger destroy-and-repair move.

4.3 Sparse effect table

A compiled landscape has many blocks and many target cells, but a single block/chain pair affects only a small number of `(year, account)` cells. The effect table stores, for each block and candidate chain, a sparse row of those cells and their values relative to the untreated block. The table also stores the empty-schedule base totals and block row starts.

The representation makes evaluation local. A move on one block is priced by the cells that block's old and new chains touch, not by iterating over the whole schedule. The sparse rows are built once and cached; the table identity includes the dataset fingerprint, candidate order, target-cell geometry, horizon, period width, and table format version. A change to weights, priorities, or targets is a pricing change, not a table rebuild.

4.4 Resident exact state

The optimizer maintains a resident state of realized totals per `(year, account)`, together with the current chain assigned to each block. Applying a move adds the new chain's contribution and subtracts the old chain's contribution. Undo restores the previous state exactly. The state is authoritative: the search uses it for the objective, and the replay path reconstructs it from the schedule through the same evaluator.

For a move on block `b` from chain `old` to chain `new`, let p be the sparse pending change to the realized account totals and let $\mathcal{T}$ be the targets it affects. The objective delta is

$$\Delta Z_b(c_{\text{old}} \to c_{\text{new}}) = \sum_{t \in \mathcal{T}} \left[\pi_t(T + p) - \pi_t(T) \right], \tag{5}$$

where π_t is the target's penalty function. For a linear level target, π_t is a per-cell penalty. For the even-flow target, π_t is a penalty over the period vector, so the delta re-evaluates the affected flow target rather than summing independent cell penalties. The cost is proportional to the pending cells plus the flow target's period vector, not to the number of blocks in the schedule. The implementation uses this identity in the local-search paths and retains a whole-schedule evaluator as a reference.

4.5 GPU population search and exact replay

The architecture separates breadth from exactness. A batched GPU population evaluates candidate-index genomes against the sparse effect table. The GPU path is used for population-scale exploration; it does not replace the exact accounting state.

The design also supports a CPU-refinement handoff: load an elite schedule into the exact resident state, apply local moves, and write the incumbent back as a GPU seed. That handoff is a design path, not a measured contribution in this paper. The 100,000-generation campaign reported below is a `ws4-gpu-ga` run with exact CPU replay; it does not contain a CPU-refinement arm or CPU-refinement counters. The handoff is therefore shown as a possible extension, not as evidence for the scaling result.

The measured division is narrower: GPU population search produces the incumbents, and the exact CPU evaluator is the replay authority. This is an engineering choice, not a claim that GPU search is universally better than CPU search.

4.6 Replay and provenance

Every reported schedule is stored as a complete block/treatment/offset assignment. The record carries the objective, seed, configuration, hardware, software, schedule path, and SHA-256. Replay loads the schedule through the compiled model and exact evaluator and compares the resulting objective with the record. A record with a mismatched hash or objective is not evidence.

The architecture therefore has two levels of reproducibility. The search is reproducible from its configuration and seed for a fixed implementation, and the reported result is independently replayable from the stored schedule even if the search process is not rerun. The second property is the one the paper relies on for its tables.

4.7 Instance and target

All experiments use the frozen TSA24/Prince George artifact described in Section 3. The compiled dataset contains 458,708 blocks; 208,255 are operable and form the search universe. The horizon is 200 years, divided into twenty 10-year periods. The target combines a weight-10,000 even-flow band with weight-100 level aspirations of 70,000,000 m^3 per period. The level aspiration is deliberately unattainable. It is a pull-up gradient, not a feasibility constraint.

4.8 Hardware and software

The GPU runs used an NVIDIA RTX PRO 6000 Blackwell Max-Q Workstation Edition (97,887 MiB) with an Intel Xeon Gold 6254 CPU at 3.10 GHz. The software was Python 3.12.3, PyTorch, and WS4 at commit `f60ce64` on the `feature/gpu-acceleration` branch. The CPU baseline used WS4 at commit `f23d450` with 32 threads; the record does not claim that commit is on main. The public artifact and its rebuild recipe are released separately; the large GPU effect table and caches are not part of the 4.66 MB artifact.

Table 1: Seed-7 scaling on the frozen TSA24 instance. Wall time includes model and cache setup; search time is the GPU search budget.

Generations	Objective	Flow term	Level term	Search / wall (s)
1,000	8,588,725.370	8,588,225.550	499.821	9.38 / 23.13
10,000	76,540.741	76,026.909	513.832	89.78 / 157.34
100,000	517.862	2.721	515.141	856.10 / 941.51

4.9 Search configurations

The GPU population size was 64. We report three budgets: 1,000, 10,000, and 100,000 generations. The 1,000- and 10,000-generation runs use seed 7 and correspond to the earlier architecture and scaling records. The 100,000-generation campaign uses seeds 1–5; seed 7 is retained as a reference run. Every GPU run starts from the same oldest-first feasible seed. The initial objective is 1,722,144,291.336651.

The CPU baseline is simulated annealing at 100,000 iterations with a nominal 30-minute limit and 32 threads, using the same instance, target, and seed. It is a secondary, untuned comparator. It performs 100,000 single proposals, whereas the GPU-GA path evaluates a population of 64 for 100,000 generations (6.4 million genome evaluations). The two budgets are therefore not equal-work or equal-time, and the comparison is reported only to show the scale of the gap.

4.10 Metrics and replay

The primary metric is the weighted objective. We also report the flow and level components separately, the realized harvest-level range, search time, wall time, and seed distribution. Every reported schedule is replayed through the exact CPU `IncrementalObjective`; the recorded objective is the replay objective, not the device objective. A record with a mismatched schedule hash or replay objective is not evidence.

A full-instance whole-schedule ablation is impractical: a single full evaluation of the 208,255-operable-block schedule is the motivating cost. We therefore report a disclosed diagnostic ablation on the first 5,000 blocks of the recorded 100k schedule. The subset is not a substitute for a full-instance ablation, but it isolates the evaluation path from the GPU population search.

5 Results

5.1 Scaling from 1,000 to 100,000 generations

Table 1 reports the seed-7 runs at the three budgets. The 1,000-generation run reduced the objective from 1,722,144,291.336651 to 8,588,725.370498 in 9.381 s of search time (23.13 s wall). The 10,000-generation run reached 76,540.740733 in 89.78 s search (157.34 s wall). The 100,000-generation run reached 517.861982 in 856.10 s search (941.51 s wall). The 100,000-generation result is not a near-zero objective: the flow term is small, but the deliberate level-aspiration term remains.

5.2 Five-seed 100,000-generation campaign

Table 2 reports the five-seed campaign at 100,000 generations. The objectives are tightly clustered: the best seed is 512.574, the worst is 518.621, the mean is 515.682, and the median is 515.938. Seed 7, at 517.862, lies inside the same range. All five schedules replay exactly. The campaign took 3,995.906 s wall for all five runs.

Table 2: Five-seed 100,000-generation campaign. All objectives are exact CPU replays.

Seed	Objective	Flow term	Level term	Realized level (M m^3/period)
1	518.621	1.331	517.290	51.21–52.25
2	512.574	0.000	512.574	51.46–52.49
3	515.938	0.000	515.938	51.31–52.32
4	514.746	0.000	514.746	51.34–52.35
5	516.530	0.076	516.454	51.26–52.29
7	517.862	2.721	515.141	51.34–52.38

The realized level is approximately 51.21–52.49M m^3 per 10-year period against the 70M aspiration. The level term is therefore the dominant part of the final objective, not a residual that the search failed to remove. The even-flow term falls from 8.59M at 1,000 generations to 2.72 at 100,000 generations; the level term remains near 499.8–517.3 because the target is deliberately above the achievable level. This is the expected maximum-even-flow behaviour of the target construction.

5.3 CPU simulated-annealing comparator

The CPU simulated-annealing run reached 335,044,976.675435 in 1,875.12 s wall (31.25 min) and stopped at the nominal 30-minute limit after 100,000 iterations. It is not converged and not a controlled baseline. Its objective is far above the GPU-GA results in Table 1. We do not interpret this as a universal architecture win: the CPU run is untuned, single-seed, and performs much less work than the GPU population. It is reported to keep the comparison honest and to show that the CPU-SA path is not a substitute for the accelerated search in this setting.

5.4 Evaluation-path ablation

On the 5,000-block diagnostic subset, a cold whole-schedule evaluation took 54.85 s; after warm-up, a whole-schedule move evaluation took 1.086 s. Exact incremental delta plus apply took 0.542 ms per move, a factor of 2,005. The final incremental objective matched the whole-schedule objective to 3.7×10^{-9}. The subset scope is recorded in `evidence/ws4-tsa24-eval-ablation.json`. This establishes the local evaluation-path effect; it does not measure GPU batched evaluation against a matched CPU batch, and it does not establish full-instance scaling.

5.5 What the results establish

The results establish four things. First, the compiled-state pipeline can run a 208,255-operable-block instance at a 100,000-generation GPU budget and produce an exact, replayable schedule. Second, the seed distribution at that budget is tight on this instance. Third, the maximum-even-flow construction produces a flat profile at a level substantially below the aspirational 70M target; the remaining level penalty is intentional. Fourth, the CPU-SA comparator is weak and does not support a controlled performance claim.

The results do not establish that the GPU path is faster per unit of exact work than a tuned CPU implementation, that the architecture is the cause of the observed quality, or that the final schedule is optimal. Those questions require the architecture ablation and matched CPU-GA/CPU-SA experiments described in the limitations.

6 Discussion

The architecture addresses a practical failure mode in applied landscape optimisation: the model, the accounting state, and the search move are often rebuilt together. Separating them has three consequences. The compiled dataset can be versioned and reused. A move can be priced exactly from a sparse effect row. The resident state can be replayed independently of the search process. These properties are valuable even when the search itself is a standard metaheuristic.

The maximum-even-flow construction is a good example of why exact accounting matters. The objective is not a single harvest target. It is a weighted compromise between a shape target and deliberately unattainable level aspirations. A search that treats the level target as a hard floor would misread the model; a search that ignores the level term would stop at a lower flat profile. The target design requires both terms to remain visible. The reported residual table is therefore part of the result, not an implementation diagnostic.

The GPU/CPU handoff is an engineering compromise. The GPU path provides population-scale exploration; the CPU path provides exact state and local refinement. The present experiments show that the combined path can reach a tight seed distribution at 100,000 generations on the TSA24 instance. They do not isolate the contribution of the GPU search from the contribution of the compiled state or the target construction. That isolation is the next experimental step.

7 Limitations and conclusion

The TSA24 instance is a public frozen artifact derived from public BC data, but there is no independent harvest-scheduling model baseline on the same instance. The paper cannot claim competitive landscape performance, state-of-the-art quality, or partner-endorsed operational impact.

The 100,000-generation result is a maximum-even-flow compromise, not a proof of the exact maximum. The 70M level aspiration is deliberately unattainable; the realized level is approximately 51.21–52.49M m^3 per period. An exact MIP proof is not practical at this scale with the available resources.

The CPU-SA comparator is untuned, single-seed, time-limited, and not equal-work. It is not a controlled baseline. The evaluation-path ablation is diagnostic rather than full-instance: it uses a 5,000-block subset and does not measure GPU batched evaluation against a matched CPU batch. Those measurements are required before the architecture claim can be separated from a local implementation-speed claim.

Within those limits, the paper supports a narrower conclusion: a compiled-state architecture can make a 208,255-block landscape search exact, replayable, and fast enough for a 100,000-generation GPU campaign, and a public artifact makes the result independently inspectable. That is the contribution. The competitive landscape claim remains open.

Data and code availability

The TSA24 benchmark artifact is available at `https://github.com/UBC-FRESH/femic-tsa24-instance/releases/tag/ws4-tsa24-evenflow-v1`. The build and replay recipe is in `BENCHMARK_ARTIFACT.md` in that repository. The paper-specific schedules and replay scripts are retained in the project evidence repository; a public evidence release remains pending maintainer approval.

References

Pete Bettinger, David Graetz, John Sessions, et al. Eight heuristic planning techniques applied to three increasingly difficult wildlife planning problems. *Silva Fennica*, 36, 2002. doi: 10.14214/sf.545.

Kevin Boston and Pete Bettinger. Combining tabu search and genetic algorithm heuristic techniques to solve spatial harvest scheduling problems. *Forest Science*, 48(1):35–46, 2002. doi: 10.1093/forestscience/48.1.35.

Christopher Lockwood and Thomas Moore. Harvest scheduling with spatial constraints: a simulated annealing approach. *Canadian Journal of Forest Research*, 23:468–478, 1993. doi: 10.1139/x93-065.